รายงานการทดลอง: Multi-threaded File Download Client/Server

Protocol, Concurrency และการเปรียบเทียบ Traditional I/O กับ NIO Native Transfer

ชื่อ-สกุล / รหัสนักศึกษา: _____

1. วัตถุประสงค์

- สร้าง Client/Server ผ่าน TCP สำหรับดาวน์โหลดไฟล์แบบหลายส่วนพร้อมกัน
- ออกแบบ protocol สำหรับขอรายชื่อไฟล์ ขอข้อมูลไฟล์ และรับส่งข้อมูลเฉพาะช่วง
- เปรียบเทียบ Traditional I/O กับ NIO/native transfer และอธิบายผลด้านเวลาและ throughput

2. การออกแบบโปรแกรม

2.1 Protocol

ใช้ header เป็นข้อความหนึ่งบรรทัด (จบด้วย `\n`) ตามด้วย payload เป็น byte ดิบ

Client ส่ง	Server ตอบ
`LIST`	`FILE <name> <size>` ทีละไฟล์ แล้วปิดด้วย `END`
`INFO <filename>`	`SIZE <bytes>` หรือ `ERROR <code> <message>`
`GET <filename> <offset> <length>`	`OK <length>` ตามด้วย byte ดิบ `length` ตัว หรือ `ERROR <code> <message>`

รหัส error ที่ใช้: 400 (คำสั่งหรือตัวเลขไม่ถูกต้อง), 404 (ไม่พบไฟล์), 416 (ช่วงข้อมูลเกินขอบเขตไฟล์)

ข้อสำคัญ: header และ payload อยู่ใน stream เดียวกัน จึงอ่าน header ทีละ byte จาก `InputStream` โดยตรง ไม่ใช้ `BufferedReader` เพราะ reader แบบมีบัฟเฟอร์จะอ่านล่วงหน้ากลืน byte ของ payload ไปด้วย ทำให้ข้อมูลเสียหาย

2.2 Server

- ใช้ `ServerSocketChannel.accept()` รับ connection แล้วส่งให้ **Virtual Thread** ดูแล 1 connection ต่อ 1 thread ผ่าน `Executors.newVirtualThreadPerTaskExecutor()`

- Virtual Thread (Java 21) เป็น thread น้ำหนักเบาที่ JVM จัดการ เมื่อติดรอ I/O JVM จะพัก thread นั้นและนำ thread จริงไปทำงานอื่น จึงรองรับหลาย connection ได้โดยเขียนโค้ดแบบ blocking ธรรมดา
- handler อ่านเฉพาะช่วงที่ร้องขอ ด้วย `RandomAccessFile.seek(offset)` แล้ว `read` ทีละ 64 KB (โหมด Traditional) หรือ `FileChannel.transferTo` (โหมด NIO)
- ตรวจความถูกต้องของ request: ชื่อไฟล์ต้องไม่มี `/`, `\`, `..` (กันการเข้าถึงนอกโฟลเดอร์), offset/length ต้องเป็นตัวเลขและอยู่ในขอบเขตไฟล์

2.3 Client

1. ส่ง `INFO` เพื่อขอขนาดไฟล์
2. สร้างไฟล์ปลายทางล่วงหน้าด้วย `RandomAccessFile.setLength(size)`
3. แบ่งไฟล์เป็นช่วงตามจำนวน worker (ค่าเริ่มต้น 10): `offset = i * chunk`, worker สุดท้ายรับส่วนที่เหลือ (`size - offset`) ช่วงข้อมูลจึงไม่ซ้อนกันและครอบคลุมทุก byte
4. แต่ละ worker เป็น Virtual Thread ที่มี connection, offset, length และ file handle/channel ของตนเอง ส่ง `GET` แล้วเขียนข้อมูลลงไฟล์ปลายทางตามตำแหน่ง offset (เขียนลง final file โดยตรง ไม่ต้อง merge part files)
5. รอทุก worker ด้วย `Future.get()` แล้วตรวจขนาดไฟล์และ SHA-256

จำนวน worker และโหมดกำหนดจาก command line: `java Client <filename> [workers=10] [nio]`

2.4 โหมดการรับส่งข้อมูล

โหมด	ฝั่ง Server	ฝั่ง Client
Traditional I/O	`RandomAccessFile.read` → `byte[]` 64 KB → `OutputStream.write`	`InputStream.read` → `byte[]` 64 KB → `RandomAccessFile.write`
NIO / native transfer	`FileChannel.transferTo(pos, count, socketChannel)`	`FileChannel.transferFrom(socketChannel, pos, count)`

ทั้งสองเมธอดของ NIO ต้องเรียกวนใน loop จนครบ เพราะไม่รับประกันว่าจะโอนครบตามที่ขอในการเรียกครั้งเดียว

Zero copy

โหมด Traditional ข้อมูลของไฟล์วิดีโอต้องถูกก๊อปหลายรอบ: ดิสก์ → Page Cache (kernel) → `byte[]` ใน JVM (user space) → socket buffer (kernel) → การ์ดเครือข่าย ส่วน `transferTo` ให้ระบบปฏิบัติการส่งข้อมูลจากไฟล์ไปยัง socket โดยตรง (เช่น `sendfile` บน Linux) ข้อมูลไม่ผ่านเข้า user space จึง

ลดการก๊อปและลดงานของ CPU และ garbage ใน heap

3. วิธีการทดลอง

- ไฟล์ทดสอบ: `video.mp4` ขนาด 2,115,126,862 bytes (ประมาณ 2017 MB) ใช้ไฟล์เดียวกันทุกกรณี
- Server และ Client รันบนเครื่องเดียวกันผ่าน localhost (พอร์ต 5000)
- 4 กรณี: Traditional 1 worker, Traditional 10 workers, NIO 1 worker, NIO 10 workers
- แต่ละกรณีรัน 3 รอบ โดยรัน warm-up 1 รอบก่อนเพื่อให้ไฟล์ต้นทางอยู่ใน Page Cache เหมือนกันทุกกรณี (ไม่นับรอบ warm-up) และลบไฟล์ปลายทางก่อนทุกรอบ
- เวลาวัดตั้งแต่เริ่มแบ่งงานให้ worker จนทุก worker เสร็จ (ไม่รวมเวลาคำนวณ hash) throughput = ขนาดไฟล์ (MB) / เวลา (วินาที)
- ตรวจความถูกต้องด้วยขนาดไฟล์และ SHA-256 เทียบกับไฟล์ต้นฉบับ (`21b1a847b2466041fa9caff65c4c6e2a46e1970fa13df8dc87130821e6899460`)
- สภาพแวดล้อม: CPU / RAM / ดิสก์ _____ , ระบบปฏิบัติการ Windows, Java _____ (ต้อง 21 ขึ้นไป)

4. ผลการทดลอง

กรณี	รอบ 1	รอบ 2	รอบ 3	เฉลี่ย (วินาที)	เฉลี่ย (MB/s)
Traditional, 1 worker	4.274 s (471.99 MB/s)	4.214 s (478.64 MB/s)	3.887 s (518.90 MB/s)	4.125	489.84
Traditional, 10 workers	3.014 s (669.31 MB/s)	2.780 s (725.50 MB/s)	2.763 s (730.15 MB/s)	2.852	708.32
NIO, 1 worker	7.581 s (266.07 MB/s)	7.562 s (266.75 MB/s)	7.610 s (265.05 MB/s)	7.584	265.96
NIO, 10 workers	3.015 s (669.02 MB/s)	2.912 s (692.78 MB/s)	2.723 s (740.67 MB/s)	2.883	700.82

ขนาดไฟล์ปลายทางตรงกับต้นฉบับทุกรอบ (2,115,126,862 bytes) และ SHA-256 ตรงกับไฟล์ต้นฉบับทุกรอบ

อัตราเร่งจากการใช้ 10 workers เทียบกับ 1 worker (เวลาเฉลี่ย):

- Traditional: 4.125 / 2.852 ≈ 1.45 เท่า
- NIO: 7.584 / 2.883 ≈ 2.63 เท่า

เปรียบเทียบ NIO กับ Traditional: ที่ 1 worker NIO ใช้เวลามากกว่าประมาณ 1.84 เท่า ที่ 10 workers ใกล้เคียงกัน (2.883 วินาที เทียบกับ 2.852 วินาที ต่างกันประมาณ 1%)

5. การวิเคราะห์ผล

5.1 ทำไม 10 workers จึงไม่เร็วขึ้น 10 เท่า

- ทุก worker ใช้ทรัพยากรร่วมกัน ได้แก่ Page Cache และดิสก์ของเครื่องเดียว (ทั้งอ่านฝั่ง Server และเขียนฝั่ง Client), loopback และ CPU ที่ต้องรันทั้ง Server และ Client พร้อมกัน เมื่อเพิ่ม worker คอขวดย้ายจากความเร็วของ connection เดียวไปเป็นความจุรวมของเครื่อง จึงแตะเพดานราว 700-740 MB/s ทั้งสองวิธี
- มีงานส่วนที่ทำแบบลำดับ ได้แก่ การสร้างไฟล์ปลายทาง การเปิด connection และการรอ worker ตัวที่ช้าที่สุด ตามกฎของ Amdahl ส่วนที่ขนานไม่ได้จำกัดอัตราเร่งสูงสุด
- Traditional 1 worker ทำได้ราว 490 MB/s อยู่แล้ว จึงเหลือที่ให้เพิ่มน้อย อัตราเร่งจึงเป็นเพียง 1.45 เท่า
- มีค่าใช้จ่ายของการจัดการ thread และ connection จำนวนมากขึ้น

5.2 ทำไม NIO จึงไม่ชนะทุกครั้ง

- ที่ 1 worker NIO ช้ากว่า Traditional อย่างสม่ำเสมอ (ทั้งสามรอบห่างกันไม่ถึง 0.1 วินาที จึงไม่ใช่ความผันผวนของเครื่อง) สมมติฐานที่น่าจะเป็นคือ `transferFrom` ฝั่ง Client ที่รับจาก socket ไม่ได้เป็น zero copy เต็มรูปแบบ JDK น่าจะอ่านผ่านบัฟเฟอร์ภายในที่เล็กกว่าบัฟเฟอร์ 64 KB ของโหมด Traditional ทำให้เรียก system call มากกว่า (สมมติฐานนี้ยังไม่ได้ทดลองแยกฝั่งเพื่อยืนยัน)
- ประโยชน์หลักของ zero copy อยู่ที่ `transferTo` ฝั่ง Server แต่การทดสอบผ่าน loopback ไม่มีการ์ดเครือข่ายจริง ส่วนที่ประหยัดได้จึงน้อย
- ที่ 10 workers คอขวดอยู่ที่ทรัพยากรร่วมของเครื่อง ความต่างของวิธีก๊อปข้อมูลจึงจางหาย ผลของสองวิธีใกล้เคียงกัน
- ขนาดบัฟเฟอร์ที่ใช้ในโหมด Traditional (64 KB) ไม่ได้เล็ก จึงเป็นคู่เทียบที่แข็งแรงพอสมควร

6. ข้อจำกัดของการทดสอบผ่าน localhost

- **Page Cache:** หลัง warm-up ไฟล์ 2 GB อยู่ใน RAM การอ่านจึงเร็วกว่าอ่านจากดิสก์จริงมาก ผลที่ได้ไม่สะท้อนความเร็วของดิสก์
- **Loopback network:** ข้อมูลไม่ได้ผ่านเครือข่ายจริง ไม่มี latency แบนด์วิดท์จำกัด หรือการสูญหายของแพ็กเก็ต จึงเห็นประโยชน์ของ zero copy กับการ์ดเครือข่ายไม่ชัด
- **Storage cache:** การเขียนไฟล์ปลายทางอาจค้างใน write-back cache ของ OS หรือ SSD ก่อนลงดิสก์จริง

- Server และ Client ใช้ CPU เครื่องเดียวกัน และทุกรอบเปิด JVM ใหม่ (มีต้นทุนเริ่มต้นและ JIT)
- ค่าที่วัดแปรผันตามสถานะเครื่องในขณะนั้น (เช่น รอบ warm-up ของ Traditional 10 workers ได้ 2.267 วินาที ขณะที่รอบวัดจริงได้ 2.76-3.01 วินาที) และทดสอบเพียง 3 รอบต่อกรณี จึงใช้ดูแนวโน้มได้ แต่ไม่ควรสรุปเกินกว่านี้

7. สรุป

- Client/Server ทำงานถูกต้องตาม protocol ทุกคำสั่ง ไฟล์ขนาด 1.96 GB ถูกดาวน์โหลดด้วย 10 workers ที่ใช้ connection, offset, length แยกกันและไม่ซ้อนกัน ขนาดและ hash ตรงกับต้นฉบับ
- การเพิ่มเป็น 10 workers เร่งความเร็วได้ 1.45-2.63 เท่า ไม่ใช่ 10 เท่า เพราะทรัพยากรร่วมและส่วนที่ทำแบบลำดับ
- NIO/native transfer ไม่ได้ชนะในทุกกรณี: ช้ากว่าที่ 1 worker และใกล้เคียงกับ Traditional ที่ 10 workers ผลขึ้นกับทรัพยากรร่วมของเครื่อง ขนาดบัฟเฟอร์ และสถานการณ์ที่วัด
- การใช้ Virtual Thread ทำให้ Server รองรับหลาย connection พร้อมกันด้วยโค้ดแบบ blocking ที่อ่านง่าย

8. วิธี compile และ run

```
javac Protocol.java Server.java Client.java

java Server            (โหมด Traditional I/O)
java Server nio        (โหมด NIO transferTo)

java Client <filename> [workers=10] [nio]
```

ตัวอย่าง:

```
java Client video.mp4 1
java Client video.mp4 10
java Client video.mp4 10 nio
```

วางไฟล์ที่ให้ดาวน์โหลดไว้ในโฟลเดอร์ `files/` ไฟล์ที่ดาวน์โหลดจะอยู่ใน `downloads/` ต้องเปิด Server โหมดเดียวกับ Client ก่อนรัน และต้องใช้ Java 21 ขึ้นไป